EnduranceLife: Advanced Physiological & Training Analytics Platform

Final Project Presentation

David Feng

School of Computing
University of Leeds

April 14, 2026

Project Vision

Problem: Most fitness platforms (like Strava) focus heavily on social features while neglecting deep physiological analytics and lifestyle correlation.

Solution: *EnduranceLife* — A purely objective, data-driven Web API designed for serious endurance athletes.

Key Goals Achieved:

- Automated ingestion of raw binary `.fit` files (Garmin/Coros).
- Complex SQL-driven analytics and pacing predictions.
- Seamless correlation of objective performance with subjective lifestyle metrics (sleep, fatigue).
- Fully secure, zero-trust data access via JWT.

System Architecture & Stack

Built on modern, robust Python technologies to ensure scalability.

Backend Framework

- **FastAPI**: High-performance async routing, automated Swagger/OpenAPI docs.
- **Pydantic V2**: Strict data validation and serialization.

Database Layer

- **SQLAlchemy 2.0**: Advanced ORM.
- **Dual-Database**: SQLite (in-memory tests/local) seamlessly switching to PostgreSQL (Render cloud production).

Client / Dashboard
↕ (REST JSON / JWT)
FastAPI Enpoints
↕ (SQL ORM)
PostgreSQL

Innovation: The Data Pipeline

Smart Ingestion (`/activities/upload`)

- Accepts native **binary** `.fit` **files**.
- Extracts bounding timestamps, distance, and time-series arrays dynamically via `fitdecode` instead of demanding tedious JSON entry.

Composite-Key Metrics (`PUT /daily-metrics/by-date`)

- A smart-upsert logic preventing the need for clients to lookup row IDs. Frontend simply dictates "today's sleep data".

Continuous Testing Data

- Included Python `scripts/` orchestrate realistic historical seed data generation (physiology, correlated metrics) to prove the analytical logic.

The Analytics Engine

Endpoints entirely offload computation to the SQL layer to prevent CPU/Memory bloat in Python.

- **Performance Records (PRs):** Evaluates fixed distance records (5K/10K) while mathematically executing **anomaly filtering** (e.g. eliminating glitched GPS paces faster than 2:30/km).
- **Predictive Physiology:** Built-in Daniels VO2-Velocity quadratic formula to estimate sustainable race paces dynamically based on the latest physical snapshot.
- **Environmental CROSS-JOINs:** Utilizes SQLAlchemy `case()` to group runs into Cold/Moderate/Hot buckets to mathematically graph cardiovascular drift.

Development Practices & Version Control

Version Control & Repository Management

- **Git / GitHub:** Project maintained via strict version control using standardized commit messages (*e.g.*, *"feat: IDOR prevention"*, *"fix: route conflict"*).
- Iterative historical log tracks the system's evolution from a bare-bones SQLite setup to a fully cloud-deployed PostgreSQL environment.

API Documentation Strategy

- Complete OpenAPI spec dynamically generated via **FastAPI / Swagger UI**.
- **Interactive Swagger:** Enables real-time browser testing, embedded `Bearer Token` injection, and direct endpoint manipulation.
- **ReDoc Extraction:** Exported to a static PDF (`api_documentation.pdf`) for offline evaluation and grading reference.

Deliverables & Report Highlights

Submission Deliverables Overview

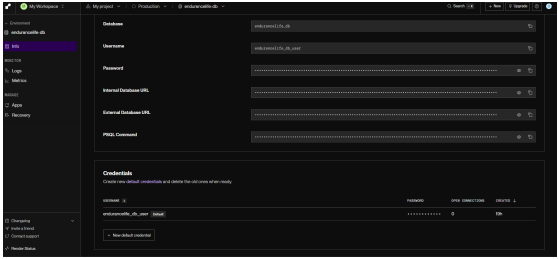
- **GitHub Repository:** Full Python source code, CI/CD scripts, and test suite.
- **API Documentation:** Hosted live and available via `docs/api_documentation.pdf`.
- **Technical Report:** Deep-dive architectural review.

Technical Report Highlights

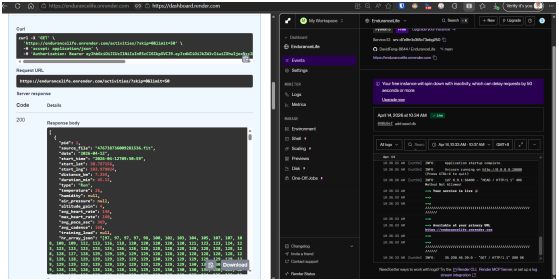
- **Design Choices:** Dual SQLite/Postgres strategy to balance local dev-speed with cloud persistence.
- **Zero-Trust Security:** Explanation of the global `Depends(get_current_user)` implementation preventing IDOR.
- **Generative AI Appendix:** Comprehensive disclosure on how tools like Gemini accelerated algorithm calibration and boilerplate generation, along with full conversational logs.

Live Cloud Deployment Evidence

Persistent PostgreSQL backend on Render



PostgreSQL Managed Instance



Server Logs & Live Swagger UI

Security & Quality Assurance (Demo transition)

Zero-Trust IDOR Prevention

- Replaced generic payload tracking (`pid=X`) with a strict `Depends(get_current_user)` architecture.
- Engine completely isolates database writes and analytic reads to the Cryptographically verified JWT token holder.

Testing

- **77 automated `pytest` cases** executed on ephemeral in-memory SQLite transactions.
- Ensures edge-cases, validation checks, and data isolation remain undefeated.

$\rightarrow$ **Proceeding to Live Swagger UI Demonstration...**

References & Datasets

Libraries & Frameworks

- **FastAPI / Pydantic V2 / SQLAlchemy ORM**: Core backend architecture.
- **fitdecode**: Binary file parsing (`https://github.com/polyvertex/fitdecode`).
- **passlib & python-jose**: JSON Web Token creation and bcrypt verification.

Datasets & API

- **Open-Meteo**: Free, open-source historical weather API (`https://open-meteo.com/`).
- **Coros FIT files**: Personal dataset generated by Coros Pace 2 smartwatch.

Theory

- **Jack Daniels' Running Formula**: Mathematical physiological estimation model.